C++ Competitive Programming Notes

Single-sheet reference: STL, built-in functions, data structures, DP and the tricks that actually show up in contests. Every code block in this document is compiled before the PDF is built, so it is safe to copy by hand.

1. Start of contest 2. Know your limits 3. C++ basics you will actually type 4. Lambdas

5. Sort, reverse, max, min and friends 6. vector and string 7. stack — LIFO 8. queue and deque — FIFO

9. Linked list 10. set and map 11. priority_queue — heap 12. Bits 13. Trees 14. Graphs

15. Essential data structures 16. Dynamic programming 17. Math and number theory 18. Strings

19. Techniques that solve half the problems 20. Geometry starter 21. Bug checklist — read this when stuck

1. Start of contest

Template

```cpp
#include <bits/stdc++.h>
using namespace std;

using ll  = long long;
using pii = pair<int,int>;
using vi  = vector<int>;

const ll  INF  = 4e18;        // for long long
const int IINF = 1e9;         // for int
const ll  MOD  = 1000000007;

void solve() {
    int n;
    cin >> n;
    vector<ll> a(n);
    for (ll &x : a) cin >> x;
    cout << accumulate(a.begin(), a.end(), 0LL) << '\n';
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int t = 1;
    cin >> t;                 // delete if single test case
    while (t--) solve();
}
```

`ios::sync_with_stdio(false); cin.tie(nullptr);` makes `cin`/`cout` fast. After it, never mix `printf`/`scanf` with `cin`/`cout`. Use `'\n'`, not `endl` (`endl` flushes and is slow in loops).

Compile flags

```
g++ -std=gnu++17 -O2 -Wall -Wextra -Wshadow -o a a.cpp
g++ -std=gnu++17 -g -fsanitize=address,undefined -o a a.cpp   # find UB / bad index
```

Reading input

```cpp
int n; ll big; double d; string word, line;
cin >> n >> big >> d >> word;   // >> skips all whitespace
cin.ignore();                   // eat the leftover newline
getline(cin, line);             // whole line, spaces included
while (cin >> n) { /* read until EOF */ }
```

Output formatting

```cpp
double x = 3.14159265;
cout << fixed << setprecision(9) << x << '\n';  // 9 decimals
cout << setw(5) << setfill('0') << 42 << '\n';  // 00042
cout << boolalpha << (1 < 2) << '\n';           // true
```

2. Know your limits

n up to	Intended complexity	Typical approach
10-12	O(n!)	brute force permutations
20-25	O(2^n), O(2^n · n)	subset / bitmask DP
100	O(n^3)	Floyd-Warshall, interval DP
1000-5000	O(n^2)	2-D DP, all pairs loops
1e5-1e6	O(n log n)	sort, set/map, segment tree
1e6-1e8	O(n)	prefix sums, two pointers, greedy
1e18	O(log n)	binary search, fast power, math

Roughly **1e8 simple operations per second**. Anything with `map`, `set` or modulo is 3-10x slower than plain array work.

Type	Max	Notes
`int`	~2.1e9	overflows constantly, be careful
`long long`	~9.2e18	the default for sums and products
`unsigned long long`	~1.8e19	wraps instead of UB
`__int128`	~1.7e38	works in g++, cannot be printed directly
`double`	~15 digits	never use `==` to compare

```cpp
int a = 100000, b = 100000;
ll bad  = a * b;              // overflow: int * int is computed as int
ll good = (ll)a * b;          // cast one side first
```

3. C++ basics you will actually type

```cpp
int arr[100] = {};            // all zeros
int grid[50][50] = {};
vector<int> v(10, -1);        // size 10, all -1
vector<vector<int>> g(5, vector<int>(3, 0));   // 5 x 3

for (int i = 0; i < 10; i++) { }
for (int x : v) { }           // read-only copy
for (int &x : v) x *= 2;      // modify in place
for (auto &[key, val] : map<string,int>{}) { }   // structured binding
```

Functions, references, structs

```cpp
void mutate(vector<int> &v) { v.push_back(1); }   // & = no copy, edits caller
int addup(const vector<int> &v) {                 // const & = no copy, read-only
    return v.empty() ? 0 : v[0];
}

struct Point {
    int x = 0, y = 0;
    int norm() const { return x * x + y * y; }
    bool operator<(const Point &o) const {        // enables sort() and set<Point>
        return x != o.x ? x < o.x : y < o.y;
    }
};
```

Always pass containers by reference. Passing a `vector` by value copies it and is a very common TLE.

pair and tuple

```
pair<int,string> p = {1, "a"};
cout << p.first << p.second;
auto [num, str] = p;                        // unpack

tuple<int,int,string> t = {1, 2, "x"};
int f = get<0>(t);
auto [i1, i2, s1] = t;
(void)f; (void)num; (void)str; (void)i1; (void)i2; (void)s1;
```

Pairs and tuples compare lexicographically, so `sort` on `vector<pair<int,int>>` sorts by `first`, then `second`, for free.

4. Lambdas

```
auto add    = [](int a, int b) { return a + b; };
int  total  = 0;
auto bump   = [&total](int x) { total += x; };      // capture by reference
auto snap   = [total](int x)  { return total + x; };// capture by value (copy)
auto all    = [&](int x) { return total + x; };     // capture everything by ref
bump(5);
(void)add(1,2); (void)snap(1); (void)all(1);
```

Use `[&]` almost always in contests: it lets the lambda see every local variable, which is what you want inside `solve()`.

Lambda as comparator

```
vector<pair<int,int>> v{{1,5},{2,3}};
sort(v.begin(), v.end(), [](const auto &a, const auto &b) {
    return a.second < b.second;             // sort by second, ascending
});
```

Recursive lambda (DFS without a global)

```
vector<vector<int>> g(5);
vector<bool> vis(5, false);
function<void(int)> dfs = [&](int u) {
    vis[u] = true;
    for (int v : g[u]) if (!vis[v]) dfs(v);
};
dfs(0);
```

C++14 alternative without `std::function` (faster):

```
vector<vector<int>> g(5);
auto dfs = [&](auto &&self, int u) -> void {
    for (int v : g[u]) self(self, v);
};
dfs(dfs, 0);
```

5. Sort, reverse, max, min and friends

Everything here lives in `<algorithm>` and takes a `[first, last)` range, so the end iterator is **one past** the last element.

```cpp
vector<int> v{4, 1, 3, 1};

sort(v.begin(), v.end());                       // ascending
sort(v.rbegin(), v.rend());                     // descending (reverse iters)
sort(v.begin(), v.end(), greater<int>());       // descending
stable_sort(v.begin(), v.end());                // keeps ties in input order
reverse(v.begin(), v.end());
rotate(v.begin(), v.begin() + 1, v.end());      // left shift by 1

int mx = *max_element(v.begin(), v.end());
int mn = *min_element(v.begin(), v.end());
int pos = max_element(v.begin(), v.end()) - v.begin();   // index of max
ll sum = accumulate(v.begin(), v.end(), 0LL);   // 0LL, not 0, or it overflows
int ones = count(v.begin(), v.end(), 1);
(void)mx; (void)mn; (void)pos; (void)sum; (void)ones;

int biggest  = max({1, 7, 3});                  // max of a list
int smallest = min(4, 9);
int clamped  = clamp(15, 0, 10);                // -> 10
swap(biggest, smallest);
(void)clamped;
```

```cpp
vector<int> v{4, 1, 3, 1};
fill(v.begin(), v.end(), 0);                    // set all to 0
iota(v.begin(), v.end(), 0);                    // 0,1,2,3,...
partial_sum(v.begin(), v.end(), v.begin());     // in-place prefix sums
nth_element(v.begin(), v.begin() + 2, v.end()); // O(n) k-th smallest

sort(v.begin(), v.end());
v.erase(unique(v.begin(), v.end()), v.end());   // remove duplicates

bool any = any_of(v.begin(), v.end(), [](int x){ return x > 2; });
bool all2 = all_of(v.begin(), v.end(), [](int x){ return x >= 0; });
(void)any; (void)all2;

// erase every 3 from an unsorted vector
v.erase(remove(v.begin(), v.end(), 3), v.end());
```

Binary search (array must be sorted)

```cpp
vector<int> v{1, 3, 3, 5, 7};
auto lo = lower_bound(v.begin(), v.end(), 3);   // first element >= 3
auto hi = upper_bound(v.begin(), v.end(), 3);   // first element  > 3
int idx = lo - v.begin();                       // index form
int cnt = hi - lo;                              // how many 3s
bool found = binary_search(v.begin(), v.end(), 5);
(void)idx; (void)cnt; (void)found;
```

For `set` and `map` use the member versions `s.lower_bound(x)`, not the free function: the free one is O(n) on node-based containers.

Permutations

```cpp
vector<int> v{1, 2, 3};
sort(v.begin(), v.end());                       // must start sorted
do {
    // visit this permutation
} while (next_permutation(v.begin(), v.end()));
```

6. vector and string

```
vector<int> v;
v.push_back(3);
v.emplace_back(4);            // constructs in place
v.pop_back();
v.size(); v.empty(); v.clear();
v.front(); v.back();
v.resize(10, 0);
v.assign(5, 7);               // 5 copies of 7
v.insert(v.begin() + 1, 9);   // O(n)
v.erase(v.begin() + 1);       // O(n)
vector<int> w = v;            // deep copy
```

`v[i]` does no bounds checking; `v.at(i)` throws. Reading `v[i]` out of range is the classic silent wrong-answer bug.

```
string s = "hello";
s += " world";
s.size(); s.substr(2, 3);           // "llo" : start 2, length 3
s.find("lo");                       // index, or string::npos
if (s.find("zz") == string::npos) { }
s.push_back('!'); s.pop_back();
reverse(s.begin(), s.end());
sort(s.begin(), s.end());           // strings sort like vectors

int num = stoi("42");
ll  big = stoll("123456789012");
string back = to_string(99);
(void)num; (void)big; (void)back;

char c = 'c';
isdigit(c); isalpha(c); islower(c); toupper(c);
int digit = '7' - '0';              // char -> int
int index = c - 'a';                // 0..25
(void)digit; (void)index;
```

Build strings with `+=` in a loop, never `s = s + x` (that copies each time).

7. stack — LIFO

```
stack<int> st;
st.push(1);
st.push(2);
int top = st.top();       // 2, does NOT remove
st.pop();                 // removes, returns nothing
bool e = st.empty();
size_t n = st.size();
(void)top; (void)e; (void)n;
```

`st.top()` on an empty stack is undefined behaviour — always check `empty()`.

Balanced brackets

```cpp
bool balanced(const string &s) {
    stack<char> st;
    for (char c : s) {
        if (c == '(' || c == '[' || c == '{') st.push(c);
        else {
            if (st.empty()) return false;
            char o = st.top(); st.pop();
            if (c == ')' && o != '(') return false;
            if (c == ']' && o != '[') return false;
            if (c == '}' && o != '{') return false;
        }
    }
    return st.empty();
}
```

Monotonic stack — next greater element

```cpp
// nxt[i] = index of the first j > i with a[j] > a[i], else -1
vector<int> next_greater(const vector<int> &a) {
    int n = a.size();
    vector<int> nxt(n, -1);
    stack<int> st;                      // indices, values decreasing
    for (int i = 0; i < n; i++) {
        while (!st.empty() && a[st.top()] < a[i]) {
            nxt[st.top()] = i;
            st.pop();
        }
        st.push(i);
    }
    return nxt;
}
```

Same skeleton solves: largest rectangle in a histogram, stock span, and "sum over all subarrays of the maximum".

8. queue and deque — FIFO

```cpp
queue<int> q;
q.push(1);
int f = q.front();     // oldest
int b = q.back();      // newest
q.pop();               // removes from the front
(void)f; (void)b;

deque<int> dq;         // push/pop at both ends, O(1), supports dq[i]
dq.push_back(1);
dq.push_front(0);
dq.pop_back();
dq.pop_front();
```

Sliding window maximum (deque of indices)

```cpp
vector<int> window_max(const vector<int> &a, int k) {
    deque<int> dq;                      // indices, values decreasing
    vector<int> res;
    for (int i = 0; i < (int)a.size(); i++) {
        while (!dq.empty() && dq.front() <= i - k) dq.pop_front();
        while (!dq.empty() && a[dq.back()] <= a[i]) dq.pop_back();
        dq.push_back(i);
        if (i >= k - 1) res.push_back(a[dq.front()]);
    }
    return res;
}
```

9. Linked list

`std::list` is a doubly linked list: O(1) insert/erase anywhere, but no `[i]` and poor cache behaviour. In contests you rarely need it — a `vector` or `deque` is usually better. Interview-style problems want the manual version.

```cpp
list<int> L{1, 2, 3};
L.push_front(0);
L.push_back(4);
auto it = L.begin();
advance(it, 2);            // no it + 2 for lists
L.insert(it, 99);          // insert before it, O(1)
L.erase(it);               // O(1)
L.sort();                  // member sort, std::sort does not work on lists
L.reverse();
L.unique();                // remove consecutive duplicates
```

Manual singly linked list

```
struct Node {
    int val;
    Node *next = nullptr;
    explicit Node(int v) : val(v) {}
};

Node* push_front(Node *head, int v) {
    Node *n = new Node(v);
    n->next = head;
    return n;
}

Node* reverse_list(Node *head) {
    Node *prev = nullptr;
    while (head) {
        Node *nxt = head->next;
        head->next = prev;
        prev = head;
        head = nxt;
    }
    return prev;                 // new head
}

// Floyd: slow/fast pointers. Also gives the middle node.
bool has_cycle(Node *head) {
    Node *slow = head, *fast = head;
    while (fast && fast->next) {
        slow = slow->next;
        fast = fast->next->next;
        if (slow == fast) return true;
    }
    return false;
}

Node* remove_value(Node *head, int v) {
    while (head && head->val == v) head = head->next;
    for (Node *cur = head; cur && cur->next; ) {
        if (cur->next->val == v) cur->next = cur->next->next;
        else cur = cur->next;
    }
    return head;
}
```

10. set and map

Container	Ordered	Duplicates	Lookup
`set`	yes	no	O(log n)
`multiset`	yes	yes	O(log n)
`unordered_set`	no	no	O(1) average
`map`	yes	keys unique	O(log n)
`multimap`	yes	yes	O(log n)
`unordered_map`	no	keys unique	O(1) average

```cpp
set<int> s;
s.insert(5);
s.insert(1);
s.count(5);                       // 0 or 1, the usual "contains" test
s.erase(5);                       // by value
if (s.find(1) != s.end()) { }     // by iterator
int smallest = *s.begin();        // min
int largest  = *s.rbegin();       // max
auto it = s.lower_bound(3);       // first >= 3, O(log n)
(void)smallest; (void)largest; (void)it;
```

```cpp
map<string,int> freq;
freq["abc"]++;                    // missing keys are created with value 0
freq.insert({"x", 5});
if (freq.count("abc")) { }
for (auto &[key, val] : freq) { } // iterated in sorted key order
freq.erase("abc");

auto it = freq.find("nope");      // ALWAYS use find() to test-then-read;
if (it != freq.end()) it->second++;   // freq["nope"] would insert a 0
```

```cpp
multiset<int> ms{1, 1, 2};
ms.erase(ms.find(1));             // erases ONE copy
ms.erase(1);                      // erases ALL copies - easy bug
```

Prefer `unordered_map` for speed, but it can be hacked to O(n) on Codeforces. Fix: add a random seed, or just use `map` if n is small.

```cpp
struct FastHash {
    size_t operator()(uint64_t x) const {
        static const uint64_t R =
            chrono::steady_clock::now().time_since_epoch().count();
        x += 0x9e3779b97f4a7c15ULL + R;
        x = (x ^ (x >> 30)) * 0xbf58476d1ce4e5b9ULL;
        return x ^ (x >> 31);
    }
};
unordered_map<uint64_t,int,FastHash> safe_map;
```

Custom key ordering:

```cpp
struct ByLen {
    bool operator()(const string &a, const string &b) const {
        return a.size() != b.size() ? a.size() < b.size() : a < b;
    }
};
set<string, ByLen> by_len;
```

The comparator must be a **strict weak ordering**: `cmp(a,a)` has to be false. Returning `<=` causes crashes inside `sort`.

11. priority_queue — heap

```
priority_queue<int> maxheap;                              // largest on top
priority_queue<int, vector<int>, greater<int>> minheap;  // smallest on top
maxheap.push(3);
minheap.push(3);
int t = maxheap.top();
maxheap.pop();
(void)t;

// heap of pairs, smallest by first
priority_queue<pii, vector<pii>, greater<pii>> pq;
pq.push({2, 100});

// custom order via lambda
auto cmp = [](const pii &a, const pii &b) { return a.second > b.second; };
priority_queue<pii, vector<pii>, decltype(cmp)> pq2(cmp);
pq2.push({1, 2});
```

There is no "decrease key". The standard trick is to push a new entry and skip stale ones when popping (see Dijkstra below).

12. Bits

```
int x = 22;
int k = 3;
bool bit  = x >> k & 1;        // read bit k
x |=  1 << k;                  // set
x &= ~(1 << k);                // clear
x ^=  1 << k;                  // flip
int low  = x & -x;             // lowest set bit as a value
bool pow2 = x && !(x & (x - 1));
(void)bit; (void)low; (void)pow2;

__builtin_popcount(x);         // number of 1 bits
__builtin_popcountll(1LL);     // ll version - use it or you lose the high bits
__builtin_clz(x);              // leading zeros  (undefined for x == 0)
__builtin_ctz(x);              // trailing zeros (undefined for x == 0)
31 - __builtin_clz(x);         // floor(log2(x))
```

Use `1LL << k` whenever `k` can be 31 or more.

Iterate subsets

```
int n = 4;
for (int mask = 0; mask < (1 << n); mask++) {
    for (int i = 0; i < n; i++)
        if (mask >> i & 1) { /* i is in the set */ }
}

int mask = 13;
for (int sub = mask; sub; sub = (sub - 1) & mask) {
    // every non-empty subset of mask, total 3^n over all masks
}
```

```
bitset<1000> bs;               // 1000 bits, 8x less memory, 64x faster ops
bs.set(5); bs.reset(5); bs.flip();
bs.count();                    // popcount
bitset<1000> other;
bs |= other; bs &= other;      // whole-array boolean ops in n/64 time
```

13. Trees

Binary tree and traversals

```
struct TNode {
    int val;
    TNode *left = nullptr, *right = nullptr;
    explicit TNode(int v) : val(v) {}
};

void inorder(TNode *r, vector<int> &out) {       // left, node, right
    if (!r) return;
    inorder(r->left, out);
    out.push_back(r->val);
    inorder(r->right, out);
}
// preorder  = node, left, right
// postorder = left, right, node

int height(TNode *r) {
    return r ? 1 + max(height(r->left), height(r->right)) : 0;
}

vector<vector<int>> level_order(TNode *root) {
    vector<vector<int>> levels;
    if (!root) return levels;
    queue<TNode*> q;
    q.push(root);
    while (!q.empty()) {
        int sz = q.size();
        levels.push_back({});
        while (sz--) {
            TNode *u = q.front(); q.pop();
            levels.back().push_back(u->val);
            if (u->left)  q.push(u->left);
            if (u->right) q.push(u->right);
        }
    }
    return levels;
}
```

An inorder traversal of a BST gives a sorted sequence — a frequent test.

```
TNode* bst_insert(TNode *r, int v) {
    if (!r) return new TNode(v);
    if (v < r->val) r->left  = bst_insert(r->left, v);
    else            r->right = bst_insert(r->right, v);
    return r;
}

bool bst_find(TNode *r, int v) {
    while (r) {
        if (r->val == v) return true;
        r = v < r->val ? r->left : r->right;
    }
    return false;
}
```

Rooted tree as a graph (what contests actually use)

```
// n nodes, n-1 edges, no cycles
vector<int> sz_of, depth_of;
vector<vector<int>> tree;

void dfs_tree(int u, int p) {
    sz_of[u] = 1;
    for (int v : tree[u]) {
        if (v == p) continue;          // "don't go back to the parent"
        depth_of[v] = depth_of[u] + 1;
        dfs_tree(v, u);
        sz_of[u] += sz_of[v];
    }
}
```

Tree diameter: BFS from any node to find the farthest node `a`, then BFS from `a`; the greatest distance found is the diameter.

14. Graphs

```
int n = 5, m = 4;
vector<vector<int>> g(n);                      // unweighted
vector<vector<pair<int,ll>>> wg(n);            // weighted: {neighbour, cost}
for (int i = 0; i < m; i++) {
    int u = 0, v = 1;
    // cin >> u >> v;  --  input is usually 1-indexed
    u--; v--;
    g[u].push_back(v);
    g[v].push_back(u);                         // drop this line if directed
}
```

BFS — shortest path in an unweighted graph

```
vector<int> bfs(const vector<vector<int>> &g, int s) {
    vector<int> dist(g.size(), -1);
    queue<int> q;
    dist[s] = 0;
    q.push(s);
    while (!q.empty()) {
        int u = q.front(); q.pop();
        for (int v : g[u]) if (dist[v] == -1) {
            dist[v] = dist[u] + 1;
            q.push(v);
        }
    }
    return dist;
}
```

DFS (iterative — recursion can overflow the stack at n = 1e5+)

```
void dfs_iter(const vector<vector<int>> &g, int s, vector<bool> &vis) {
    vector<int> st{s};
    vis[s] = true;
    while (!st.empty()) {
        int u = st.back(); st.pop_back();
        for (int v : g[u]) if (!vis[v]) {
            vis[v] = true;
            st.push_back(v);
        }
    }
}
```

Counting connected components: loop over all nodes, run BFS/DFS from each unvisited one, count how many times you started.

Dijkstra — non-negative weights, O(m log n)

```
vector<ll> dijkstra(const vector<vector<pair<int,ll>>> &g, int s) {
    vector<ll> d(g.size(), INF);
    priority_queue<pair<ll,int>, vector<pair<ll,int>>,
                   greater<pair<ll,int>>> pq;
    d[s] = 0;
    pq.push({0, s});
    while (!pq.empty()) {
        auto [du, u] = pq.top(); pq.pop();
        if (du > d[u]) continue;            // stale entry, skip
        for (auto [v, w] : g[u]) if (du + w < d[v]) {
            d[v] = du + w;
            pq.push({d[v], v});
        }
    }
    return d;
}
```

0-1 BFS — every weight is 0 or 1, O(n + m)

```
vector<int> bfs01(const vector<vector<pii>> &g, int s) {
    vector<int> d(g.size(), IINF);
    deque<int> dq;
    d[s] = 0;
    dq.push_back(s);
    while (!dq.empty()) {
        int u = dq.front(); dq.pop_front();
        for (auto [v, w] : g[u]) if (d[u] + w < d[v]) {
            d[v] = d[u] + w;
            if (w) dq.push_back(v); else dq.push_front(v);
        }
    }
    return d;
}
```

Floyd-Warshall — all pairs, n <= 400

```
void floyd(vector<vector<ll>> &d) {          // d[i][i]=0, else INF if no edge
    int n = d.size();
    for (int k = 0; k < n; k++)              // k MUST be the outer loop
        for (int i = 0; i < n; i++)
            for (int j = 0; j < n; j++)
                if (d[i][k] < INF && d[k][j] < INF)
                    d[i][j] = min(d[i][j], d[i][k] + d[k][j]);
}
```

Bellman-Ford — negative weights, detects negative cycles

```
bool bellman(int n, const vector<tuple<int,int,ll>> &edges,
             int s, vector<ll> &d) {
    d.assign(n, INF);
    d[s] = 0;
    for (int it = 0; it < n - 1; it++)
        for (auto [u, v, w] : edges)
            if (d[u] < INF && d[u] + w < d[v]) d[v] = d[u] + w;
    for (auto [u, v, w] : edges)             // one more pass: still improving?
        if (d[u] < INF && d[u] + w < d[v]) return false;  // negative cycle
    return true;
}
```

Topological sort (Kahn) — DAG ordering / cycle detection

```
vector<int> topo(const vector<vector<int>> &g) {
    int n = g.size();
    vector<int> indeg(n, 0), order;
    for (int u = 0; u < n; u++) for (int v : g[u]) indeg[v]++;
    queue<int> q;
    for (int i = 0; i < n; i++) if (!indeg[i]) q.push(i);
    while (!q.empty()) {
        int u = q.front(); q.pop();
        order.push_back(u);
        for (int v : g[u]) if (--indeg[v] == 0) q.push(v);
    }
    return (int)order.size() == n ? order : vector<int>{};  // empty = cycle
}
```

Bipartite check (2-colouring)

```
bool bipartite(const vector<vector<int>> &g, vector<int> &col) {
    col.assign(g.size(), -1);
    for (int s = 0; s < (int)g.size(); s++) {
        if (col[s] != -1) continue;
        col[s] = 0;
        queue<int> q; q.push(s);
        while (!q.empty()) {
            int u = q.front(); q.pop();
            for (int v : g[u]) {
                if (col[v] == -1) { col[v] = col[u] ^ 1; q.push(v); }
                else if (col[v] == col[u]) return false;   // odd cycle
            }
        }
    }
    return true;
}
```

15. Essential data structures

DSU / union-find — O(α(n)) per operation

```
struct DSU {
    vector<int> p, sz;
    explicit DSU(int n) : p(n), sz(n, 1) { iota(p.begin(), p.end(), 0); }
    int find(int x) { return p[x] == x ? x : p[x] = find(p[x]); }
    bool unite(int a, int b) {
        a = find(a); b = find(b);
        if (a == b) return false;              // already connected
        if (sz[a] < sz[b]) swap(a, b);
        p[b] = a;
        sz[a] += sz[b];
        return true;
    }
    bool same(int a, int b) { return find(a) == find(b); }
};
```

Uses: connected components, cycle detection in an undirected graph, Kruskal.

Kruskal — minimum spanning tree

```
ll kruskal(int n, vector<tuple<ll,int,int>> edges) {   // {weight, u, v}
    sort(edges.begin(), edges.end());
    DSU dsu(n);
    ll total = 0;
    for (auto [w, u, v] : edges)
        if (dsu.unite(u, v)) total += w;
    return total;
}
```

Fenwick / BIT — prefix sums with point updates, O(log n)

```
struct BIT {
    int n;
    vector<ll> t;
    explicit BIT(int n) : n(n), t(n + 1, 0) {}
    void add(int i, ll v) { for (++i; i <= n; i += i & -i) t[i] += v; }
    ll sum(int i) {                             // sum of [0, i]
        ll s = 0;
        for (++i; i > 0; i -= i & -i) s += t[i];
        return s;
    }
    ll range(int l, int r) {
        return l > r ? 0 : sum(r) - (l ? sum(l - 1) : 0);
    }
};
```

Segment tree (iterative) — any associative operation

```
struct SegTree {
    int n;
    vector<ll> t;
    static const ll ID = 0;                     // identity of op
    static ll op(ll a, ll b) { return a + b; }  // swap for min/max/gcd
    explicit SegTree(int n) : n(n), t(2 * n, ID) {}
    void set(int i, ll v) {
        for (t[i += n] = v; i > 1; i >>= 1) t[i >> 1] = op(t[i], t[i ^ 1]);
    }
    ll query(int l, int r) {                    // inclusive [l, r]
        ll a = ID, b = ID;
        for (l += n, r += n + 1; l < r; l >>= 1, r >>= 1) {
            if (l & 1) a = op(a, t[l++]);
            if (r & 1) b = op(t[--r], b);
        }
        return op(a, b);
    }
};
```

For min queries set `ID = LLONG_MAX` and `op = min`. Range **update** needs lazy propagation — if updates are "add v to a range" and you only query at the end, use a difference array instead.

Sparse table — O(1) static range min/max

```
struct Sparse {
    vector<vector<int>> t;
    vector<int> lg;
    explicit Sparse(const vector<int> &a) {
        int n = a.size(), K = 1;
        while ((1 << K) <= n) K++;
        lg.assign(n + 1, 0);
        for (int i = 2; i <= n; i++) lg[i] = lg[i / 2] + 1;
        t.assign(K, vector<int>(n));
        t[0] = a;
        for (int k = 1; k < K; k++)
            for (int i = 0; i + (1 << k) <= n; i++)
                t[k][i] = min(t[k-1][i], t[k-1][i + (1 << (k-1))]);
    }
    int query(int l, int r) {                   // inclusive
        int k = lg[r - l + 1];
        return min(t[k][l], t[k][r - (1 << k) + 1]);
    }
};
```

16. Dynamic programming

DP = recursion + memory. Write it in four steps:

1. **State** — what do I need to know to finish the job? That is your index set.
2. **Transition** — how does a state depend on smaller states?
3. **Base case** — the smallest states, filled by hand.
4. **Answer** — which state (or max over states) is the result?

Top-down (memo) is easier to derive; bottom-up (table) is faster and has no stack limit. Both are fine.

Fibonacci — memo vs table

```
vector<ll> memo(100, -1);
ll fib(int n) {
    if (n <= 1) return n;
    ll &res = memo[n];                  // reference trick: read and write once
    if (res != -1) return res;
    return res = fib(n - 1) + fib(n - 2);
}

ll fib_table(int n) {
    vector<ll> dp(n + 2);
    dp[0] = 0; dp[1] = 1;
    for (int i = 2; i <= n; i++) dp[i] = dp[i-1] + dp[i-2];
    return dp[n];
}
```

Kadane — maximum subarray sum

```
ll max_subarray(const vector<ll> &a) {
    ll best = a[0], cur = a[0];
    for (size_t i = 1; i < a.size(); i++) {
        cur  = max(a[i], cur + a[i]);   // extend, or start fresh here
        best = max(best, cur);
    }
    return best;
}
```

0/1 knapsack — each item at most once

```
ll knapsack(int W, const vector<int> &wt, const vector<ll> &val) {
    vector<ll> dp(W + 1, 0);            // dp[w] = best value at capacity w
    for (size_t i = 0; i < wt.size(); i++)
        for (int w = W; w >= wt[i]; w--)   // DESCENDING = use item once
            dp[w] = max(dp[w], dp[w - wt[i]] + val[i]);
    return dp[W];
}
```

Make the inner loop **ascending** and each item can be used unlimited times (unbounded knapsack). That one-character difference is the whole distinction.

Coin change — fewest coins, and number of ways

```
int min_coins(const vector<int> &coins, int target) {
    vector<int> dp(target + 1, IINF);
    dp[0] = 0;
    for (int v = 1; v <= target; v++)
        for (int c : coins) if (c <= v)
            dp[v] = min(dp[v], dp[v - c] + 1);
    return dp[target] >= IINF ? -1 : dp[target];
}

// combinations (coins outer) — permutations if you swap the loops
ll count_ways(const vector<int> &coins, int target) {
    vector<ll> dp(target + 1, 0);
    dp[0] = 1;
    for (int c : coins)
        for (int v = c; v <= target; v++)
            dp[v] = (dp[v] + dp[v - c]) % MOD;
    return dp[target];
}
```

Subset sum — can we hit exactly `target`?

```
bool subset_sum(const vector<int> &a, int target) {
    vector<char> dp(target + 1, 0);
    dp[0] = 1;
    for (int x : a)
        for (int v = target; v >= x; v--)
            if (dp[v - x]) dp[v] = 1;
    return dp[target];
}
```

With `bitset` this becomes one line and runs 64x faster: `bs |= bs << x;` then test `bs[target]`.

LCS — longest common subsequence

```
int lcs(const string &a, const string &b) {
    int n = a.size(), m = b.size();
    vector<vector<int>> dp(n + 1, vector<int>(m + 1, 0));
    for (int i = 1; i <= n; i++)
        for (int j = 1; j <= m; j++)
            dp[i][j] = a[i-1] == b[j-1]
                     ? dp[i-1][j-1] + 1
                     : max(dp[i-1][j], dp[i][j-1]);
    return dp[n][m];
}
```

Edit distance

```cpp
int edit_distance(const string &a, const string &b) {
    int n = a.size(), m = b.size();
    vector<vector<int>> dp(n + 1, vector<int>(m + 1));
    for (int i = 0; i <= n; i++) dp[i][0] = i;      // delete everything
    for (int j = 0; j <= m; j++) dp[0][j] = j;      // insert everything
    for (int i = 1; i <= n; i++)
        for (int j = 1; j <= m; j++)
            dp[i][j] = a[i-1] == b[j-1] ? dp[i-1][j-1]
                     : 1 + min({dp[i-1][j],      // delete
                                dp[i][j-1],      // insert
                                dp[i-1][j-1]});  // replace
    return dp[n][m];
}
```

LIS — longest increasing subsequence in O(n log n)

```cpp
int lis(const vector<int> &a) {
    vector<int> tail;                   // tail[k] = smallest end of a length
    for (int x : a) {                   // k+1 increasing subsequence
        auto it = lower_bound(tail.begin(), tail.end(), x);
        if (it == tail.end()) tail.push_back(x);
        else *it = x;
    }
    return tail.size();
}
```

`lower_bound` gives strictly increasing; `upper_bound` gives non-decreasing.

Grid paths with obstacles

```cpp
ll grid_paths(const vector<vector<int>> &blocked) {
    int n = blocked.size(), m = blocked[0].size();
    vector<vector<ll>> dp(n, vector<ll>(m, 0));
    dp[0][0] = blocked[0][0] ? 0 : 1;
    for (int i = 0; i < n; i++)
        for (int j = 0; j < m; j++) {
            if (blocked[i][j]) { dp[i][j] = 0; continue; }
            if (i) dp[i][j] += dp[i-1][j];
            if (j) dp[i][j] += dp[i][j-1];
        }
    return dp[n-1][m-1];
}
```

Bitmask DP — travelling salesman, n <= 20

```
ll tsp(const vector<vector<ll>> &c) {
    int n = c.size(), full = 1 << n;
    vector<vector<ll>> dp(full, vector<ll>(n, INF));
    dp[1][0] = 0;                        // start at node 0, only 0 visited
    for (int mask = 1; mask < full; mask++)
        for (int u = 0; u < n; u++) {
            if (dp[mask][u] == INF || !(mask >> u & 1)) continue;
            for (int v = 0; v < n; v++) {
                if (mask >> v & 1) continue;
                int nm = mask | 1 << v;
                dp[nm][v] = min(dp[nm][v], dp[mask][u] + c[u][v]);
            }
        }
    ll best = INF;
    for (int u = 1; u < n; u++) best = min(best, dp[full-1][u] + c[u][0]);
    return best;
}
```

DP on trees — maximum weight independent set

```
vector<vector<int>> t_adj;
vector<ll> w_of;
vector<array<ll,2>> tdp;                 // [0] = u unused, [1] = u used

void tree_dp(int u, int p) {
    tdp[u][0] = 0;
    tdp[u][1] = w_of[u];
    for (int v : t_adj[u]) {
        if (v == p) continue;
        tree_dp(v, u);
        tdp[u][0] += max(tdp[v][0], tdp[v][1]);
        tdp[u][1] += tdp[v][0];          // neighbours must stay unused
    }
}
```

Common DP states: `dp[i]` (prefix), `dp[i][j]` (two sequences, or index+capacity), `dp[l][r]` (intervals), `dp[mask]` (subsets), `dp[i][j][k]` (add one more constraint), `dp[u][0/1]` (trees).

17. Math and number theory

```
__gcd(12, 18);                           // 6
ll a = 12, b = 18;
ll lcm_ab = a / __gcd(a, b) * b;         // divide FIRST to avoid overflow
abs(-5); llabs(-5LL); fabs(-5.0);
(ll)sqrtl(1e18);                         // then fix off-by-one by hand
(void)lcm_ab;

// ceil division for positive numbers, no floating point
ll x = 7, y = 2;
ll up = (x + y - 1) / y;                 // 4
(void)up;

// true modulo for possibly negative values
int m = 5;
int r = ((-7 % m) + m) % m;
(void)r;
```

Integer division truncates toward zero, so `-7 / 2 == -3`, not `-4`.

Modular arithmetic

```
ll pw(ll b, ll e, ll m) {                    // fast power, O(log e)
    ll r = 1 % m;
    b %= m;
    if (b < 0) b += m;
    while (e > 0) {
        if (e & 1) r = r * b % m;
        b = b * b % m;
        e >>= 1;
    }
    return r;
}

ll inv(ll b, ll m) { return pw(b, m - 2, m); }   // m prime, b % m != 0
```

Add: `(a + b) % MOD`. Subtract: `(a - b + MOD) % MOD`. Multiply: cast to `long long` first — `1e9 * 1e9` does not fit in an `int`.

Sieve and factorisation

```
vector<int> primes_upto(int n) {
    vector<bool> comp(n + 1, false);
    vector<int> pr;
    for (int i = 2; i <= n; i++) if (!comp[i]) {
        pr.push_back(i);
        for (ll j = (ll)i * i; j <= n; j += i) comp[j] = true;
    }
    return pr;
}

vector<pair<ll,int>> factor(ll x) {          // O(sqrt x)
    vector<pair<ll,int>> f;
    for (ll p = 2; p * p <= x; p++) {
        if (x % p) continue;
        int c = 0;
        while (x % p == 0) { x /= p; c++; }
        f.push_back({p, c});
    }
    if (x > 1) f.push_back({x, 1});          // leftover prime
    return f;
}

vector<ll> divisors(ll x) {                  // O(sqrt x), unsorted
    vector<ll> d;
    for (ll i = 1; i * i <= x; i++) if (x % i == 0) {
        d.push_back(i);
        if (i != x / i) d.push_back(x / i);
    }
    return d;
}
```

Binomial coefficients mod p

```
struct Comb {
    vector<ll> f, fi;
    explicit Comb(int n) : f(n + 1), fi(n + 1) {
        f[0] = 1;
        for (int i = 1; i <= n; i++) f[i] = f[i-1] * i % MOD;
        fi[n] = pw(f[n], MOD - 2, MOD);
        for (int i = n; i > 0; i--) fi[i-1] = fi[i] * i % MOD;
    }
    ll C(int n, int k) const {
        if (k < 0 || k > n) return 0;
        return f[n] * fi[k] % MOD * fi[n-k] % MOD;
    }
};
```

Handy identities: sum 1..n = `n(n+1)/2`; number of subsets = `2^n`; number of pairs = `n(n-1)/2`; `nCk = nC(n-k)`.

18. Strings

Prefix function / KMP — pattern matching in O(n + m)

```
vector<int> prefix_function(const string &s) {
    int n = s.size();
    vector<int> pi(n, 0);
    for (int i = 1; i < n; i++) {
        int j = pi[i-1];
        while (j > 0 && s[i] != s[j]) j = pi[j-1];
        if (s[i] == s[j]) j++;
        pi[i] = j;
    }
    return pi;
}

vector<int> find_all(const string &txt, const string &pat) {
    vector<int> pi = prefix_function(pat + '\1' + txt), res;
    int m = pat.size();
    for (int i = m + 1; i < (int)pi.size(); i++)
        if (pi[i] == m) res.push_back(i - 2 * m);
    return res;
}
```

`n - pi[n-1]` is the smallest period of the string — a very common trick.

Z-function

```
vector<int> z_function(const string &s) {    // z[i] = lcp(s, s[i..])
    int n = s.size();
    vector<int> z(n, 0);
    z[0] = n;
    for (int i = 1, l = 0, r = 0; i < n; i++) {
        if (i < r) z[i] = min(r - i, z[i - l]);
        while (i + z[i] < n && s[z[i]] == s[i + z[i]]) z[i]++;
        if (i + z[i] > r) { l = i; r = i + z[i]; }
    }
    return z;
}
```

Hashing — compare any two substrings in O(1)

```
struct Hash {
    static const ll M = (1LL << 61) - 1;
    vector<__int128> h, p;
    explicit Hash(const string &s) {
        static ll B = 0;
        if (!B) {                           // random base beats anti-hash tests
            mt19937_64 rng(chrono::steady_clock::now()
                           .time_since_epoch().count());
            B = 131 + rng() % 1000;
        }
        int n = s.size();
        h.assign(n + 1, 0);
        p.assign(n + 1, 1);
        for (int i = 0; i < n; i++) {
            h[i+1] = (h[i] * B + s[i]) % M;
            p[i+1] = p[i] * B % M;
        }
    }
    ll get(int l, int r) const {            // inclusive
        __int128 v = (h[r+1] - h[l] * p[r-l+1]) % M;
        return (ll)((v % M + M) % M);
    }
};
```

Trie

```
struct Trie {
    struct Node {
        array<int,26> nxt;
        int cnt = 0;
        Node() { nxt.fill(-1); }
    };
    vector<Node> t{Node()};
    void insert(const string &s) {
        int u = 0;
        for (char ch : s) {
            int c = ch - 'a';
            if (t[u].nxt[c] == -1) { t[u].nxt[c] = t.size(); t.push_back({}); }
            u = t[u].nxt[c];
        }
        t[u].cnt++;
    }
    bool contains(const string &s) {
        int u = 0;
        for (char ch : s) {
            u = t[u].nxt[ch - 'a'];
            if (u == -1) return false;
        }
        return t[u].cnt > 0;
    }
};
```

Palindrome check: `s == string(s.rbegin(), s.rend())`, or two pointers from both ends.

19. Techniques that solve half the problems

Prefix sums — range sum in O(1)

```
vector<ll> build_prefix(const vector<ll> &a) {
    vector<ll> p(a.size() + 1, 0);
    for (size_t i = 0; i < a.size(); i++) p[i+1] = p[i] + a[i];
    return p;
}
// sum of a[l..r] inclusive = p[r+1] - p[l]
```

2-D prefix sums

```
struct Prefix2D {
    vector<vector<ll>> s;
    explicit Prefix2D(const vector<vector<ll>> &a) {
        int n = a.size(), m = n ? a[0].size() : 0;
        s.assign(n + 1, vector<ll>(m + 1, 0));
        for (int i = 0; i < n; i++)
            for (int j = 0; j < m; j++)
                s[i+1][j+1] = a[i][j] + s[i][j+1] + s[i+1][j] - s[i][j];
    }
    ll query(int r1, int c1, int r2, int c2) {       // inclusive corners
        return s[r2+1][c2+1] - s[r1][c2+1] - s[r2+1][c1] + s[r1][c1];
    }
};
```

Difference array — O(1) range add, build once at the end

```
struct Diff {
    vector<ll> d;
    explicit Diff(int n) : d(n + 1, 0) {}
    void add(int l, int r, ll v) { d[l] += v; d[r+1] -= v; }
    vector<ll> finish() {
        vector<ll> a(d.size() - 1);
        ll run = 0;
        for (size_t i = 0; i + 1 < d.size(); i++) a[i] = (run += d[i]);
        return a;
    }
};
```

Two pointers — longest window with sum <= K (non-negative values)

```
int longest_window(const vector<int> &a, ll K) {
    int best = 0, l = 0;
    ll sum = 0;
    for (int r = 0; r < (int)a.size(); r++) {
        sum += a[r];
        while (sum > K) sum -= a[l++];
        best = max(best, r - l + 1);
    }
    return best;
}
```

Binary search on the answer

Works when "is `x` good?" is monotone: false, false, ..., true, true.

```cpp
ll first_true(ll lo, ll hi, const function<bool(ll)> &ok) {
    while (lo < hi) {
        ll mid = lo + (hi - lo) / 2;         // NOT (lo+hi)/2, that overflows
        if (ok(mid)) hi = mid; else lo = mid + 1;
    }
    return lo;                               // smallest x with ok(x) true
}
```

Classic uses: minimum capacity / speed / time to finish, maximise the minimum gap, "can we do it in `x` days?".

Coordinate compression

```cpp
vector<int> compress(vector<int> a) {        // values -> ranks 0..k-1
    vector<int> s = a;
    sort(s.begin(), s.end());
    s.erase(unique(s.begin(), s.end()), s.end());
    for (int &x : a) x = lower_bound(s.begin(), s.end(), x) - s.begin();
    return a;
}
```

Greedy interval scheduling

```cpp
int max_non_overlapping(vector<pii> iv) {    // sort by RIGHT endpoint
    sort(iv.begin(), iv.end(), [](const pii &a, const pii &b) {
        return a.second < b.second;
    });
    int cnt = 0, end = INT_MIN;
    for (auto [l, r] : iv) if (l >= end) { cnt++; end = r; }
    return cnt;
}
```

Grid movement

```cpp
const int dx[] = {-1, 1, 0, 0};              // add {-1,-1,1,1} / {-1,1,-1,1}
const int dy[] = {0, 0, -1, 1};              // for 8 directions
int n = 5, m = 5, x = 0, y = 0;
for (int k = 0; k < 4; k++) {
    int nx = x + dx[k], ny = y + dy[k];
    if (nx < 0 || ny < 0 || nx >= n || ny >= m) continue;
    // visit (nx, ny)
}
```

Randomness

```cpp
mt19937_64 rng(chrono::steady_clock::now().time_since_epoch().count());
ll r = rng() % 100;                          // never use rand() on Codeforces
shuffle(vector<int>{}.begin(), vector<int>{}.end(), rng);
(void)r;
```

20. Geometry starter

```
struct P {
    ll x, y;
    P operator-(const P &o) const { return {x - o.x, y - o.y}; }
    ll cross(const P &o) const { return x * o.y - y * o.x; }
    bool operator<(const P &o) const { return x != o.x ? x < o.x : y < o.y; }
};

// > 0 counter-clockwise turn, < 0 clockwise, 0 collinear
ll orient(const P &a, const P &b, const P &c) { return (b-a).cross(c-a); }

ll dist2(const P &a, const P &b) {           // squared: stays exact
    ll dx = a.x - b.x, dy = a.y - b.y;
    return dx * dx + dy * dy;
}

// twice the polygon area (shoelace); works for any simple polygon
ll area2(const vector<P> &p) {
    ll s = 0;
    for (size_t i = 0; i < p.size(); i++)
        s += p[i].cross(p[(i + 1) % p.size()]);
    return llabs(s);
}
```

Keep geometry in integers whenever you can — compare squared distances instead of calling `sqrt`.

21. Bug checklist — read this when stuck

- **Overflow.** `int * int` overflows even when stored in a `ll`. Cast first.
- **Uninitialised state between test cases.** Clear every global/vector inside the loop, not once at the start.
- **`endl` in a big loop.** Use `'\n'`.
- **Wrong index base.** Input is usually 1-indexed; decide once and subtract.
- **`v[i]` out of range.** Silent garbage. Rerun with `-fsanitize=address,undefined`.
- **Recursion depth.** DFS at n = 1e5 can blow the stack; go iterative.
- **`map[key]` in a condition.** It inserts. Use `count` or `find`.
- **`ms.erase(value)`** on a multiset deletes every copy. Use `erase(find(v))`.
- **Comparator not a strict weak ordering.** `<=` in a comparator crashes.
- **Comparing doubles with `==`.** Use `fabs(a - b) < 1e-9`.
- **Forgot `% MOD`** somewhere in a chain, or subtracted without `+ MOD`.
- **Printing `__int128` directly.** Cast to `ll` or write your own printer.
- **Reading with `>>` after `getline`** and vice versa. `cin.ignore()`.
- **Edge cases:** n = 1, empty input, all equal values, negative numbers, the maximum allowed value, a disconnected graph, a self-loop.

Stress testing (when the logic looks right but the verdict is wrong)

```
for i in $(seq 1 1000); do
  ./gen $i > in.txt
  ./brute < in.txt > out_brute.txt
  ./fast  < in.txt > out_fast.txt
  if ! diff -q out_brute.txt out_fast.txt > /dev/null; then
    echo "MISMATCH on test $i"; cat in.txt; break
  fi
done
```

Debug print

```cpp
#define dbg(x) cerr << #x << " = " << (x) << '\n'
int answer = 5;
dbg(answer);                    // prints: answer = 5
```

`cerr` is not judged, so debug output there is harmless — but remove it anyway, it costs time on large inputs.